Federated Reinforcement Learning Implemented Across a Large-Scale Distributed System

Engineering Notebook

Savanna Coffel
Harvard University
sdcoffel@college.harvard.edu

Ian Moore
Harvard University
ian_moore@college.harvard.edu

1 Introduction

The use of machine learning to take advantage of financial markets has been around for a while. Most quantitative research on Wall Street now involves utilizing machine learning to leverage predictive power to profit from the stock market. Federated learning is a relatively new area of machine learning research shown to be useful on a distributed system, as it allows for multiple clients to contribute to training a global model while preserving each client's data privacy. In this project, we were interested in using Federated Reinforcement Learning to train clients to make profitable trades on a simulated stock market.

This notebook will help us keep track of our attempts to explore how such a framework can be leveraged to tackle real-world problems that are reliant on today's distributed systems. We explain the process of building this system and training the model, and why we have decided that Federated RL, and more generally, RL itself, is *not* an ideal framework for this kind of scenario.

2 04/15/25: Vanilla System Design

In our system design, there is one server that interfaces with and mediates multiple clients. We chose this design because it is both the most straightforward way to implement the vanilla system and preserve individual client privacy concerning stock trading, and it allows us to easily aggregate the global model on a single, central server. The three main aspects of this design are the server, client, and stock simulator scripts.

2.1 Server

The server is the universal source of truth and manages all stock data and clients' portfolios. As a result, clients will only be able to see their portfolios by directly interfacing with the server. This preserves client privacy and is a much simpler way of keeping track of different clients' data. This setup will also allow us to aggregate all the incoming weights from individual clients' local training and periodically send model updates to all clients once the aggregation protocol is complete.

To manage all the underlying data, the server maintains and periodically updates three `.txt` files:

- `stocks.txt`: contains stock names and price/share. For simplicity in training, we maintain five different stocks.
- `currencies.txt`: contains information on currency conversion rates
- `clients.txt`: contains a dictionary of all registered clients on the server and their portfolios

Preprint. Under review.

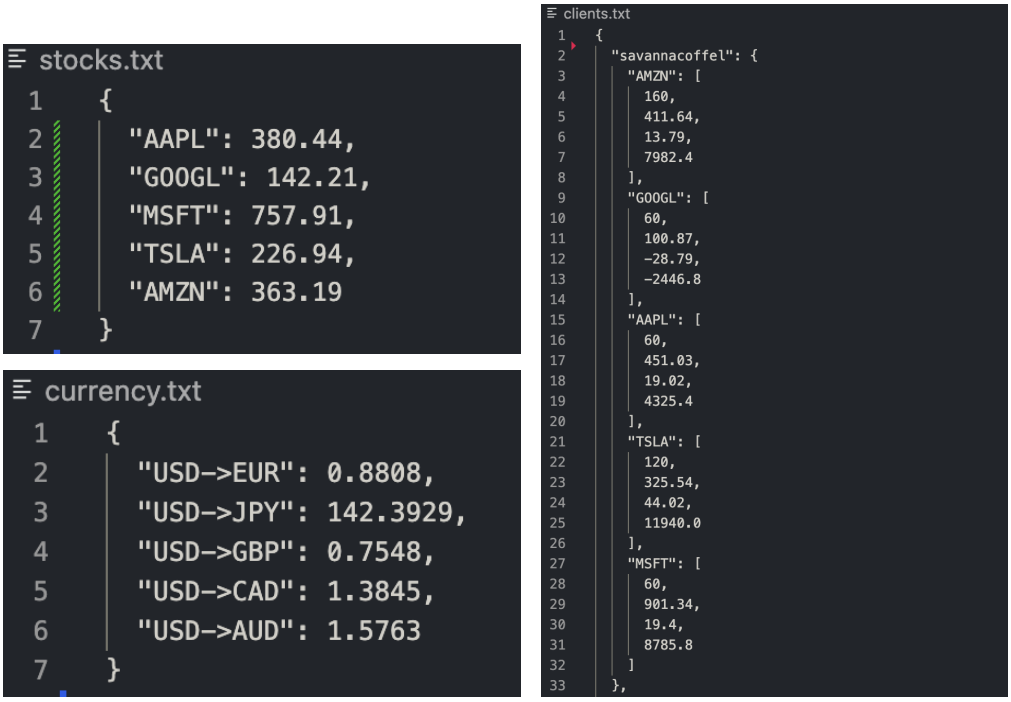

Figure 1: Underlying data maintained by the server.

We save the state of these files when the server is shut down and always load the most recent version when firing up the server, so we can achieve persistent storage here.

Note: As of 04/24/25, we do not really use `currency.txt`. We care more about the training results than trading between currencies and the unnecessary complications that could be introduced.

2.2 Client

When a client wants to see their updated portfolio, they need to send a request to the server, in which case the server will parse the request and then return to the client their corresponding updated dictionary in `clients.txt`. Before we implemented any sort of RL, we tested this by manually providing the keyword `portfolio`, prompting the server to return to the client an updated portfolio:

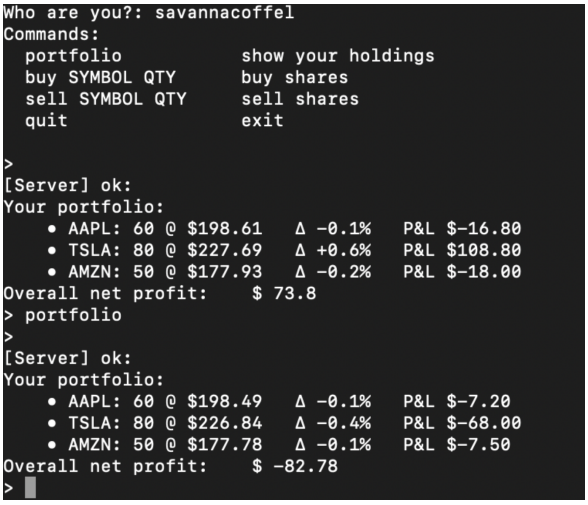

Figure 2: Example of a client's POV when requesting portfolio updates.

2.3 Market

We simulate the market and changes in stock prices via a script called `stockpricemanager.py`, which runs separately from the client and server scripts. Its primary function is to update the stock prices in `stocks.txt` to simulate a changing stock market. The stock prices are updated according to changes that follow Geometric Brownian Motion (GBM), a continuous stochastic process that we chose to model the market. The differential equation for GBM is given by:

$$dS_t = \mu S_t dt + \sigma S_t dW_t \tag{1}$$

Where W_t is an equation that represents Brownian motion, and μ, σ are constants representing the percentage drift and percentage volatility. For our simulation, we use a solution to this differential equation given by:

$$S_{t+dt} = S_t \exp\left(\left(\mu - \frac{1}{2}\sigma^2\right) dt + \sigma\sqrt{dt}\, Z\right) \tag{2}$$

`stockpricemanager.py` continually updates `stocks.txt` according to the price fluctuations dictated by (2). As clients request to see changes in their portfolio, the server pulls that data and distributes updates.

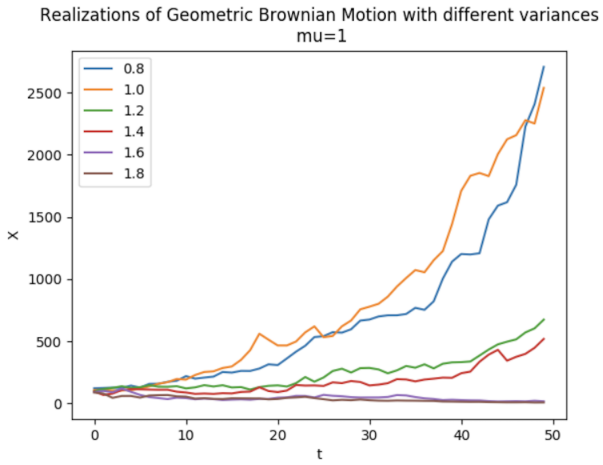

Figure 3: Realizations of GBM with different hyperparameters. *Source: Wikipedia*

There are other models that we could have chosen, but GBM is realistic enough and provides enough random noise into our model to create a satisfactory environment for training. In our simulation, price updates happen faster than in the real stock market, but because we wanted our simulation and analytics to run quickly, we decided this was an appropriate tradeoff to make.

```
Stock Price Manager firing up...
[Mon Apr 28 15:21:21 2025] Updated prices: {'AAPL': 374.0, 'GOOGL': 139.65, 'MSFT': 694.6, 'TSLA': 236.77, 'AMZN': 333.3}
[Mon Apr 28 15:21:26 2025] Updated prices: {'AAPL': 365.91, 'GOOGL': 132.42, 'MSFT': 769.53, 'TSLA': 216.22, 'AMZN': 362.27}
[Mon Apr 28 15:21:31 2025] Updated prices: {'AAPL': 417.49, 'GOOGL': 111.49, 'MSFT': 761.9, 'TSLA': 247.28, 'AMZN': 386.13}
[Mon Apr 28 15:21:36 2025] Updated prices: {'AAPL': 446.24, 'GOOGL': 120.9, 'MSFT': 854.66, 'TSLA': 228.64, 'AMZN': 410.21}
[Mon Apr 28 15:21:41 2025] Updated prices: {'AAPL': 458.33, 'GOOGL': 127.41, 'MSFT': 947.73, 'TSLA': 240.45, 'AMZN': 463.89}
[Mon Apr 28 15:21:46 2025] Updated prices: {'AAPL': 468.61, 'GOOGL': 124.83, 'MSFT': 963.42, 'TSLA': 257.96, 'AMZN': 442.35}
[Mon Apr 28 15:21:51 2025] Updated prices: {'AAPL': 532.81, 'GOOGL': 118.48, 'MSFT': 996.92, 'TSLA': 255.7, 'AMZN': 486.41}
[Mon Apr 28 15:21:56 2025] Updated prices: {'AAPL': 490.68, 'GOOGL': 118.26, 'MSFT': 945.63, 'TSLA': 255.76, 'AMZN': 507.84}
[Mon Apr 28 15:22:01 2025] Updated prices: {'AAPL': 505.49, 'GOOGL': 125.56, 'MSFT': 913.67, 'TSLA': 282.02, 'AMZN': 498.37}
```

Figure 4: Market simulation in action. The time between updates is tweakable - as of 04/28/25, we have it set to 5 seconds in between updates.

3 04/20/25-04/29/25: Testing Suite

Throughout the building of this project, we added both unit and integration tests at regular intervals to test new functionalities and features of our codebase.

3.1 Unit Tests

We have 37 unit tests covering 95% of our codebase (excluding integration tests). Our goal was to reach $\geq 90\%$ coverage in each of the client, server, and price simulation scripts.

Name	Stmts	Miss	Cover
client.py	223	21	91%
integrationtests.py	113	113	0%
server.py	180	7	96%
stockpricemanager.py	38	1	97%
unittests.py	531	13	98%
TOTAL	1085	155	86%

Figure 5: Breakdown of unit test coverage

3.2 Integration Tests

We have three integration tests.

- Simulated server and client interaction with vanilla stock trading, without any RL, to test the server-client flow.
- A test of the federated learning flow, to ensure that we are correctly passing weights between the server and clients and updating both local and global models accordingly.
- A test that verifies `stockpricemanager.py` is correctly updating `stocks.txt`, and that the server is processing these updates correctly.

4 04/20/25: Federated RL Framework

4.1 Our RL Setting

We first need to formalize this problem as a valid, well-defined reinforcement learning problem.

We distill all of our observations down to a single value: at each time step, we observe Δp_t, the change in price from the previous time step $t-1$ to the current time step t. We also have the number of shares of stock we have as a state variable. Let H_t denote the number of shares of stock we have at a given time step t. Then at time t the state $s_t = \{\Delta p_t, H_t\}$.

We then take an action. We define our action space to be

- Action 1: Buy a share of the stock. This has no effect (is equivalent to action 0) if we already have 200 shares of stock. We enforce 200 shares as the maximum number of shares we can have of a given stock. Deterministically, this increases the number of shares held by 1.
- Action -1: Sell a share of the stock. This has no effect (is equivalent to action 0) if we have 0 shares of stock, since in this case we have no shares to sell. Deterministically, this decreases the number of shares held by 1.
- Action 0: Hold the existing number of shares and do nothing.

The rewards are given by the underlying GBM process defined above. The reward at each time step is the net profit (or loss) given by the purchase or sale of a share, if one was purchased/sold.

This is thus a well-defined, undiscounted, infinite-horizon Markov Decision Process.

4.2 Our RL Algorithm, Without Federation

We rely on a simple linear model with three weights for our RL algorithm. We compute our predicted expected profit $\hat{R}$ by:

$$\hat{R} = w \cdot (1, \Delta p, a)$$

where $\cdot$ represents the dot product, Δp is the change in price (state), and a is the action taken by the agent. Realize that the first entry of w represents a bias term. (Realize that Δp is an element of the state, so our RL algorithm has access to this.)

Our algorithm then acts with respect to the action computed to have the optimal predicted expected profit. On each client, we use a simple update rule for the weights:

$$w \leftarrow \alpha(R - \hat{R})x$$

where R is the actual profit and $\hat{R}$ is the predicted profit. This is a simple policy gradient style update, and we should expect this over several iterations to converge onto the optimal set of weights.

Note from a learning theoretic standpoint that we imply the assumption that this model class (linear functions of this form) is sufficiently expressive to capture the true dynamics of the underlying environment, i.e., the GBM process determining the prices. We believe for our purposes in this basic setup that while this is a strong assumption, this is an acceptable simplification as it will give us a setup to test communication in our federated learning setup, which is what we are really interested in. However, were this system to be deployed in a real-life trading application, it is likely worth considering what other classes of functions may be better and whether we need more expressivity in the class of functions we choose.

This simple choice of model may also be well supported by the reinforcement learning literature. Since we are seeking a single model that works across a spectrum of stocks, we do not want to overfit to the noise generation process of any one stock. This is a classic bias-variance tradeoff problem. To allow sufficient generalizability, we want to inject enough bias into the model to make sure it does not overfit onto the noise generation process of any single stock. The implication of this is that in the next section, when we federate learning, this should protect the model from overfitting on a single client, thus stabilizing the aggregation and averaging of weights by the central server across the clients.

4.3 How We Can Federate

The principled/theoretical foundation for our federation scheme is simple. We allow each client to operate an RL agent that interacts with the RL environment, which in our case is the trading server. So the clients are all placing trades individually.

The client then trains its own local model, fully parameterized by the set of weights w described above. As the individual client interacts with the trading environment- i.e., by issuing trade BUY/SELL (or HOLD) orders to the server as described above, it can update its weights as described with the update rule in the prior section.

We are concerned with efficient communication and orchestration/management of the distributed system as our principal development in this project. Therefore, we take a simple approach to federating learning in this setting. We have a centralized training management server that orchestrates training. This server is responsible for orchestrating the federated learning of a single set of weights that works for all stocks we are trading. We want a generalizable model capable of trading any stock (drawn from a set distribution of stocks that behave a certain way, consistent with the learning theory in this space).

Each time step, we have all the clients send their weights to the centralized server. The centralized server then aggregates all of the weights sent by the clients, averages them, and then sends the new averaged aggregate weights back to all the clients.

We pondered a peer-to-peer setup, but ultimately determined that this would be too complex. We find that generally same bandwidth and latency issues arise whether we use a centralized server or a peer-to-peer design — the peers or the centralized server must have sufficient bandwidth to ingest all weights from all clients, and if we are moving data across geographies then either we have peers located in a different geography or a server located in a different geography, thus introducing the same type of latency concerns. In the end, these network constraints cannot be circumvented, but only

managed by our distributed system. The added simplicity of having a central server aggregate the weights once rather than it being replicated over many peers makes this design choice very appealing to us.

This simple federation setup should result in a single set of weights generalizing across the various stocks we are trading, and gives us a tractable way to test our distributed system. Moreover, the simplicity here is a strength, supported by ideas in learning theory, as we discussed with the bias-variance tradeoff in the last section.

5 04/28/25: Results

We have implemented Federated RL and updated clients with the `autotrade` feature. This allows for completely hands-free trading; the RL-powered clients now have complete control over the trading process. After implementing this, we have seen some interesting results. We discuss these results and our conclusions in this section.

Before beginning our discussion, we outline key assumptions we have made regarding the RL-client and the stock trading environment:

- **Stock prices aren't guaranteed to get better:** We're not working with ETFs or any portfolios that guarantee a positive ROI over time - these are stocks that fluctuate according to the whims of the market, so we should expect to see more chaotic behavior.
- **The RL-client's decisions are heavily influenced by the underlying GBM fluctuations:** Reinforcement learning is a very *reactive* algorithm, where the weights are updated as a result of the RL-client's decisions to buy or sell. This model has no way of predicting where the stock price will be next, it can only make trades based on the buy/sell weights assigned to it from the server *at the present moment*. Therefore, the RL-client's decisions will necessarily follow the market determined by the GBM model.
- **Fluctuations in net profit will drastically increase over time:** The longer we let the RL-client run, the more shares it is going to eventually buy of a given stock, meaning that the price fluctuations are going to become increasingly dramatic. To mitigate this, we have put an upper limit on the number of shares that can be bought of a given stock at 200.

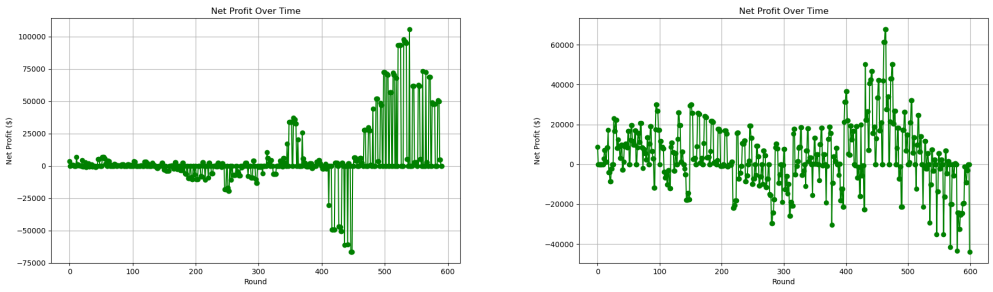

Figure 6: Some examples of changes in net profit during an RL-client's trading session.

To normalize training in our simulation, we gave every newly registered client 10 shares of each stock. We let the client run for a certain number of rounds, and then terminated the simulation with `Ctrl+c` after a given period. We chose to do this because, in a realistic trading setting, we reasoned that the RL clients would run autonomously, but human traders would be monitoring and ultimately making the final decision about when to "cash out" - that is, terminate the client and save the final profit amount.

We noticed, not unsurprisingly, that the longer we let the client run, the more drastic a drift we observed in the net profit field. Over time, as shares tended towards the upper limit of 200, their percentage changes in price had a larger effect on the overall net profit, compared to stocks with smaller amounts of shares.

This tells us something interesting and concerning. To maximize net profit, the RL-client is choosing to buy the maximum amount of shares of a stock possible, then waiting until these drastic price fluctuations produce a ridiculously high peak. Essentially, the RL-client decides that the best way to make the highest amount of net profit is by trading "riskily" - beefing up shares in one or two stocks and taking advantage of the drastic fluctuations in net profit to force the net profit field to jump as high as possible. This means that while our RL-client is *technically* solving the problem statement - maximizing the net profit - it is doing so in a remarkably unstable way - and overfitting.

6 04/28/25-04/30/25: Discussion and Analytics

Overwhelmingly, the behavior of our RL-client is to overload high-fluctuating shares and wait for a favorable shift in these shares to generate a high net profit. We observe three metrics during our training that we discuss, which both provide evidence for and explain this behavior. For all three metrics, we simulate a new test client, which we have named `laurel`.

6.1 Net Profit

Despite its instability, the RL client is capable of maximizing the net profit by taking advantage of market fluctuations for heavily owned shares. However, this is not a desirable trading strategy in the real world.

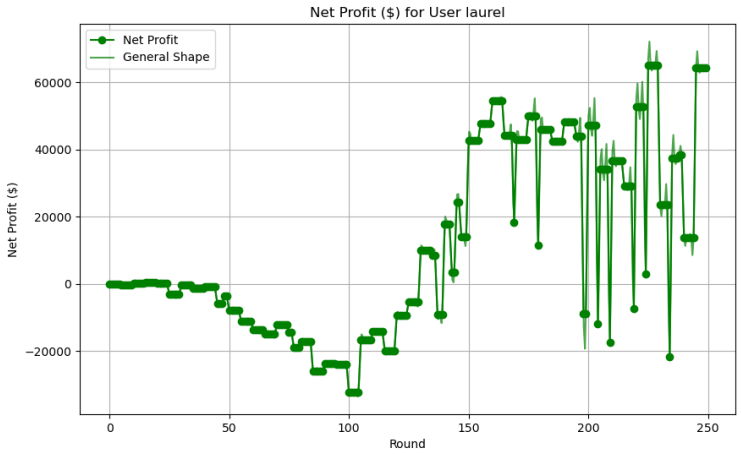

Figure 7: Net profit after 250 rounds of trading.

Why is the RL-client deciding to do this? One of the biggest reasons has to do with how our model's update rule allows the weights to drift towards whichever feature delivers the biggest payoff most recently, which, in this case, are those drastic fluctuations in the model. That is, when a sudden price spike generates a massive reward, the corresponding weight shoots upward to match it. On the next round, this weight drives the policy to take an even larger position. In summary, our agent has no method for regularization or risk control, and is overfitting to those one-off spikes to generate massive spikes in profit.

The RL-client decides to chance high-reward outliers, instead of trading stably, because the model reinforces those disproportionately high weights when a trade happens to be profitable. This gives us the rapidly fluctuating profits in Figure 7.

6.2 Loss

As market fluctuations become more drastic, any notion of equilibrium or learning stability becomes nonexistent. This is supplemented by the jump in loss as the RL-client progresses. As the fluctuations in profit become more drastic, the MSE diverges further and further from the gradient, inducing a massive spike in our loss curves, even as the net profit field climbs. This demonstrates that while

the RL-client is learning which reward system gives it the biggest payoff, it is doing so in a highly unstable way.

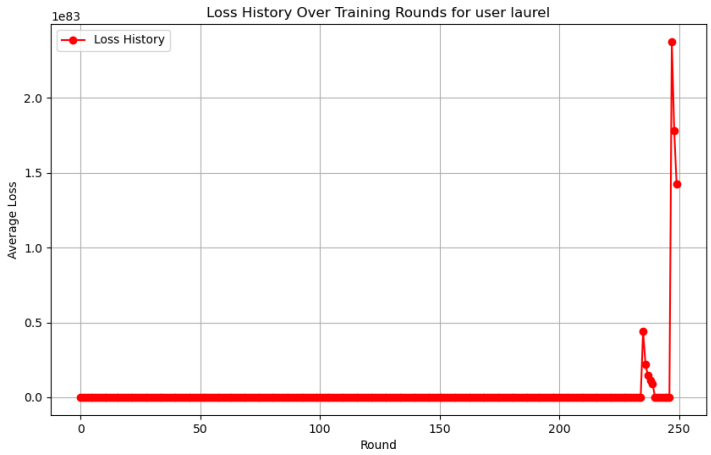

Figure 8: Loss after 250 rounds of trading.

6.3 Accuracy

We measured accuracy by keeping a running total of whether a decision made by the RL-client was a "good decision", i.e., whether the decision to buy or sell a stock was actually profitable to us. We average the percentage of correct guesses over each epoch.

Our accuracy plots are not promising. They demonstrate that the accuracy rate fluctuates wildly, and there is no convergence where the model approaches an equilibrium and "learns" what the right trade is, reinforcing the instability of the model and the risky rewards system that it is following:

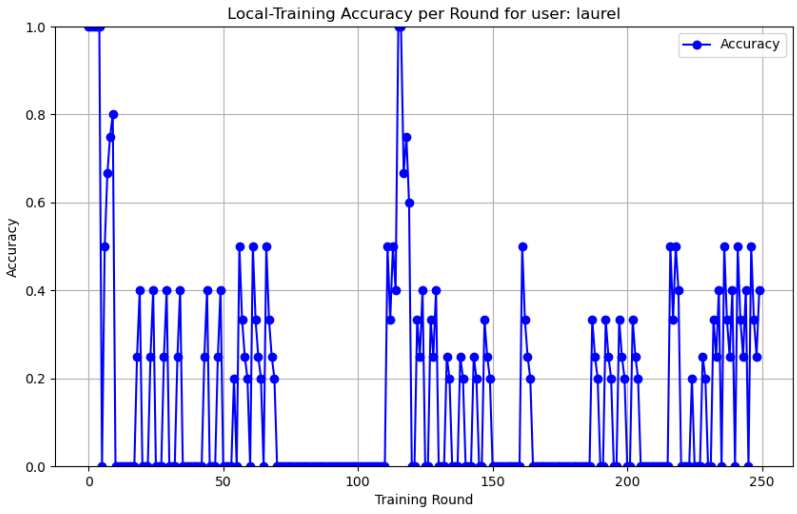

Figure 9: Accuracy after 250 rounds of trading.

Besides the instability of the rewards system, which we have previously discussed, we believe there are two other reasons why our accuracy is fluctuating so wildly:

- At the end of the day, our underlying stock market simulation is inherently *random* and changes rapidly, undergoing significant shifts in the percentages of rising or falling stock prices. This is not a typical setting where an RL model thrives, as it thrives in environments where the underlying state and reward signals do not fluctuate as drastically.
- To make the most amount of net profit, the RL-client has decided that it doesn't *need* to be accurate all the time, it only needs to buy up as many stock shares as possible and wait until the maximum fluctuation in the market. In a more stable trading approach, we would see a gradual increase in accuracy instead of the peaks and troughs shown in the above graph.

7 04/30/25: Conclusion and Final Thoughts

Federated Learning is *hard*, and Federated Reinforcement Learning is far from the best framework for something like stock trading. While yes, this framework does *technically* solve our problem statement, it is not by any means a solid machine learning framework in this context. We have discerned that the reward mechanism in this scenario is simply too skewed towards risky, unstable, high-profit swings in a client's portfolio, which is not conducive to any sort of meaningful reinforcement learning. This tells us, predominantly, that *this model is not stable*. While reinforcement learning can absolutely be used on a distributed system for other tasks, we are of the opinion, after this project, that leveraging Federated RL for a task that requires predictive power, like stock market trades, is not the way to go.

It's not all doom and gloom, though - we learned so much from this project, and building a stock trading simulator without any machine learning is already quite the feat for a distributed systems project. We would love to continue exploring this in the future by potentially leveraging more predictive-style models of Federated Learning, and we think that there are many more avenues to explore here. We investigated interesting behavior regarding RL that we had not previously seen, where the methodology behind RL was not entirely productive regarding our desired goal. So, although Federated RL was not the best framework for stock trading, we learned a lot about its intricacies and semantics through this project.

The authors would like to thank Professor James Waldo and the teaching staff of CS2620 for an amazing semester. We learned so much from this class.